Vídeo 6.2: Testando também o que dá errado

Duração-alvo: 6 min | **Conceito:** testes de sucesso e de falha | **O aluno sai sabendo:** escrever testes para o cadastro, a validação, a exclusão e as rotas que não existem.

Antes de gravar: partir do código do vídeo 6.1.

[0:00] Abertura

Na tela: título do vídeo.

Fala: “Testar quando tudo vai bem é metade do trabalho. A outra metade é testar o que acontece quando dá errado: livro que não existe, dados inválidos, endereço desconhecido. Quem usa a API vai errar, e precisamos garantir que ela responde direito.”

[0:40] Cada teste se vira sozinho

Na tela: uma frase: “cada teste cria o que precisa”.

Fala: “Uma regra de ouro: cada teste deve criar o que precisa e não depender de outro teste. Assim, se um falhar, os outros continuam confiáveis, e a ordem não importa.”

[1:15] Testes de cadastro

Na tela: criar o arquivo tests/livros.test.js e escrever o código abaixo, com o cursor acompanhando cada trecho.

```
process.env.ARQUIVO_BANCO = ':memory:';

const request = require('supertest');
const app = require('../src/app');
const { fecharBanco } = require('../src/db');

afterAll(async () => {
  await fecharBanco();
});

function cadastrarLivro(dados) {
  return request(app).post('/livros').send(dados);
}

describe('cadastro de livros', () => {
  test('cadastra um livro válido e devolve 201', async () => {
        const resposta = await cadastrarLivro({
        titulo: 'Dom Casmurro',
        autor: 'Machado de Assis'
        });

        expect(resposta.status).toBe(201);
    expect(resposta.body.titulo).toBe('Dom Casmurro');
        expect(resposta.body.id).toBeDefined();
```

```
  });

  test('recusa um livro sem título e devolve 400', async () => {
        const resposta = await cadastrarLivro({ autor: 'Machado de Assis' });

        expect(resposta.status).toBe(400);
    expect(resposta.body.erro).toBe('Dados inválidos');
  });
});
```

Fala: “O começo é igual ao do teste anterior: banco temporário, Supertest, app e fechamento do banco. Depois, uma função de ajuda, cadastrarLivro, que faz um POST com os dados que a gente passar, para não repetir esse trecho em todo teste. O primeiro teste cadastra um livro válido e espera 201, o título certo e um número de id. O segundo é o que testa o erro: manda um livro sem título e espera 400 e a mensagem de dados inválidos. É esse teste que garante que o nosso porteiro continua no lugar.”

[3:00] Consulta, exclusão e rota desconhecida

Na tela: acrescentar os blocos abaixo ao mesmo arquivo.

```
describe('consulta e exclusão', () => {
  test('devolve 404 para um livro que não existe', async () => {
        const resposta = await request(app).get('/livros/9999');

        expect(resposta.status).toBe(404);
  });

  test('exclui um livro e depois não o encontra mais', async () => {
        const criado = await cadastrarLivro({
        titulo: 'O Cortiço',
        autor: 'Aluísio Azevedo'
        });
        const id = criado.body.id;

        const exclusao = await request(app).delete(`/livros/${id}`);
        expect(exclusao.status).toBe(204);

        const busca = await request(app).get(`/livros/${id}`);
        expect(busca.status).toBe(404);
  });
});

describe('rotas desconhecidas', () => {
  test('devolve 404 em formato JSON', async () => {
        const resposta = await request(app).get('/estante-que-nao-existe');

        expect(resposta.status).toBe(404);
        expect(resposta.body.erro).toBe('Rota não encontrada');
```

```
  });
});
```

Fala: “Agora mais três testes. O primeiro pede um livro de número 9999 e espera 404. O segundo conta uma história completa: cadastra um livro, guarda o número dele, exclui, confere que veio 204, busca de novo e confere que agora veio 404. Repare que ele cria o próprio livro, sem depender de nenhum outro teste. O terceiro pede um endereço que não existe e confere o 404 no formato padronizado que montamos no módulo passado.”

[4:45] Rodando e provando o valor

Na tela: rodar `npm test` e mostrar o resumo verde. Depois, tirar o `validarLivro` da rota POST em `src/rotas/livros.js`, rodar de novo e mostrar o teste de recusa falhando. Desfazer a mudança.

Fala: “Rodo tudo: verde. E agora a prova do valor. Vou tirar o validarLivro da rota POST e rodar de novo. O teste que recusa livro sem título falha: em vez do 400 esperado, a API devolve outra coisa. Sem esse teste, a gente só descobriria o problema quando um dado ruim já estivesse no banco. Desfaço a mudança, e tudo volta a ficar verde.”

[5:30] Resumo

Na tela: uma frase: o teste do erro vale tanto quanto o do sucesso.

Fala: “Resumindo: testamos o cadastro, a validação, a exclusão e as rotas que não existem. Cada teste cria o que precisa. E o teste do erro vale tanto quanto o do sucesso. No próximo vídeo vamos documentar a API, para que outras pessoas consigam usá-la.”

Exercício sugerido: escrever um teste para o `PUT` que confirme que alterar um livro que não existe devolve 404.